

TraceCoder 代码完全导读

这份文档面向第一次接触编程智能体的读者。读完后，你应该能回答下面这些问题：

1. 大语言模型为什么能够“读取文件、修改代码、运行测试”？
2. 模型返回一次工具调用后，程序如何校验、执行并把结果送回模型？
3. TraceCoder 如何防止路径越界、无限循环、重复调用和未验证就宣布完成？
4. CLI 和 Web GUI 为什么能够复用同一套 Agent 核心？
5. Web 多轮对话如何保存上一轮消息，“新对话”为什么能够真正清空上下文？
6. 测试如何在没有真实 API key、没有真实模型的情况下验证整个 Agent 循环？

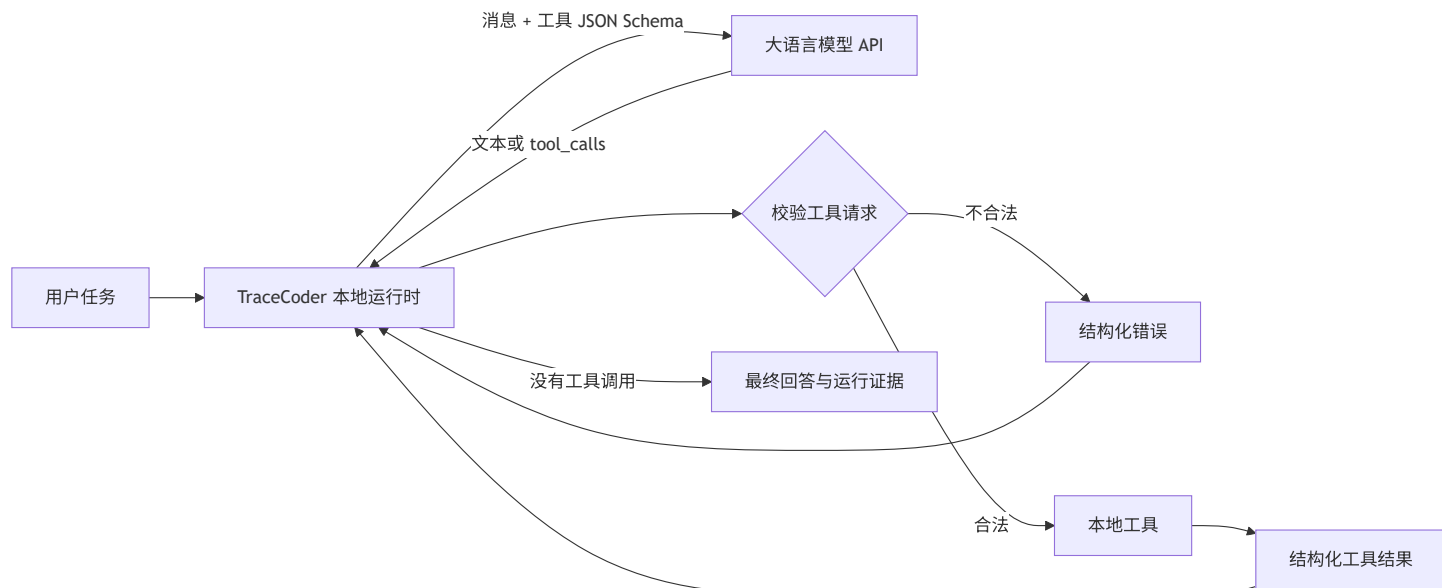
建议先完整阅读第 1~6 节建立整体认识，再按兴趣阅读工具、安全、Web 或测试部分。

1. 先理解：编程智能体到底是什么

普通聊天程序通常只进行一次“用户提问 -> 模型回答”。编程智能体多了一层由我们自己实现的运行时循环：

1. 程序把用户任务、历史消息和工具说明发送给模型。
2. 模型不一定立刻给最终答案，它可以返回“我要调用某个工具”。
3. 本地程序校验工具名和参数。
4. 本地程序执行工具，例如读取文件或运行测试。
5. 工具结果被追加到消息历史，再次发送给模型。
6. 重复以上过程，直到模型不再调用工具，或运行时触发终止条件。

因此，模型本身并没有直接获得电脑权限。真正拥有文件和进程访问能力的是 TraceCoder 的本地 Python 代码。模型只能提出结构化请求，本地运行时决定是否接受并执行。



这里有三个责任边界：

参与者	负责什么	不负责什么
大语言模型	理解任务、选择工具、生成参数、根据结果规划下一步	不直接读写磁盘，不直接启动进程
TraceCoder	管理历史、校验参数、执行工具、审批命令、记录轨迹、决定何时停止	不替模型编写具体解决方案
操作系统	真正执行文件 I/O 和子进程	不理解自然语言任务

这正是本项目的核心价值：不是给现成 Agent 套一个界面，而是自己实现中间这层 Harness。

2. 项目目标与边界

TraceCoder 是一个从零实现的单智能体、本地执行、可追踪编程 Agent。项目没有使用 LangChain、LlamaIndex、OpenAI Agents SDK、Claude Agent SDK、AutoGen 等 Agent 框架，也没有使用服务端托管的 Code Interpreter 或 Files API。

项目自己实现了：

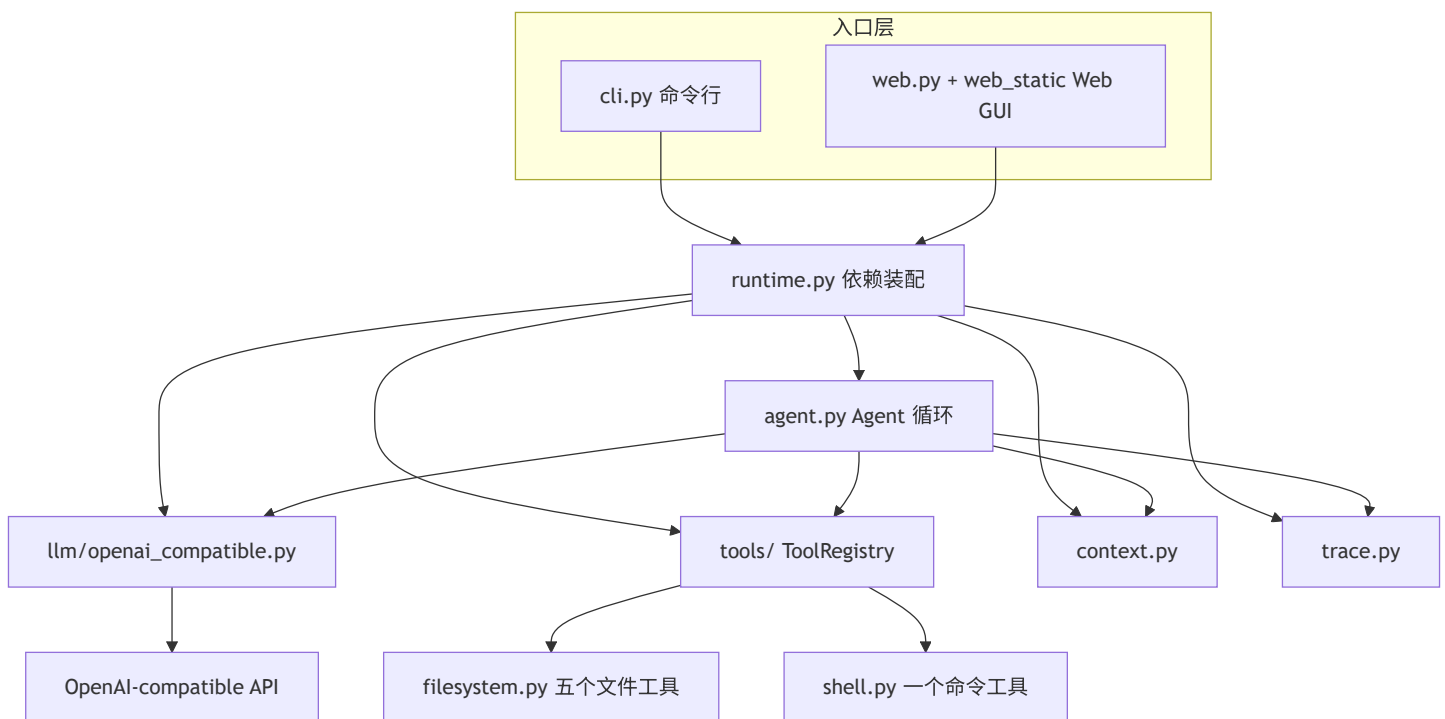
- 对话消息与多轮上下文管理；
- OpenAI-compatible tool calling 解析；
- 工具 JSON Schema、参数校验与本地分发；
- 文件工具的工作区边界；
- 命令审批、超时和输出上限；
- 模型—工具循环与确定性终止；

- 修改后的验证状态；
- JSONL 事件轨迹和凭据脱敏；
- CLI 与本地 Web GUI；
- Web 多轮会话、文件上传、轮询、停止和审批；
- 不依赖真实 API 的自动化测试。

它不是操作系统沙箱，也不是完整 IDE。命令一旦被用户批准，就拥有当前用户本来就有的主机权限。这一限制在设计中被明确记录，而不是被隐藏。

3. 总体架构

项目采用“入口层、运行时装配层、核心循环、基础设施”分层。CLI 与 Web 的交互方式不同，但都通过 `runtime.py` 创建同一个 Agent。



各层依赖方向是单向的：

- `domain.py` 定义最基础的数据结构，不依赖具体 SDK。
- Agent 依赖抽象的 `ModelClient`，不直接导入 OpenAI 客户端。
- OpenAI-compatible 细节被限制在一个适配器中。
- CLI 和 Web 只负责输入输出，不复制 Agent 决策逻辑。
- 文件和命令副作用集中在 `tools/` 中。

这种结构的好处是可测试：测试可以把真实模型替换成 `FakeModelClient`，而不需要修改 Agent。

4. 目录结构

```
coding_agent/
├─ pyproject.toml          # Python 包、依赖、CLI 入口、Ruff/mypy/pytest 配置
├─ README.md              # 对外项目首页
├─ README.txt             # 题目要求的 1000 汉字以内提交说明
├─ .env                   # 本地凭据，不进入 Git
├─ .github/workflows/ci.yml # Windows/Linux 自动测试
├─ .devcontainer/devcontainer.json # Codespaces 环境和端口转发
├─ src/tracecoder/
│   ├─ domain.py          # 领域数据类型与运行结果
│   ├─ config.py          # .env 和环境变量配置
│   ├─ agent.py           # 核心模型-工具循环
│   ├─ context.py         # 上下文预算与压缩
│   ├─ runtime.py         # 组装模型、工具、轨迹和 Agent
│   ├─ trace.py           # 脱敏 JSONL 轨迹
│   ├─ cli.py             # run / trace / web 命令
│   ├─ web.py             # FastAPI、会话、审批、上传和运行协调
│   └─ llm/
│       ├─ base.py        # 模型协议
│       └─ openai_compatible.py # Chat Completions 适配器
│   └─ tools/
│       ├─ registry.py    # 工具注册、Schema 和参数校验
│       ├─ filesystem.py  # 工作区文件工具
│       └─ shell.py       # 审批后的命令执行
│   └─ web_static/
│       ├─ index.html     # 页面语义结构
│       ├─ styles.css     # 页面布局和样式
│       ├─ markdown.js    # 安全 Markdown 渲染
│       └─ app.js         # 浏览器状态、API 调用和轮询
└─ tests/
    ├─ fakes.py           # 假模型
    ├─ test_agent_loop.py # 核心循环测试
    ├─ test_*             # 配置、工具、上下文、轨迹、CLI、Web 测试
    ├─ markdown_smoke.cjs # Markdown 安全冒烟测试
    └─ web_ui_smoke.cjs   # 执行真实前端脚本的 DOM 冒烟测试
```

5. 最基础的数据合同： domain.py

在写循环前，项目先定义模型、工具和运行时之间交换什么数据。这里使用

```
@dataclass(frozen=True, slots=True) :
```

- frozen=True：对象创建后不能随意修改，减少状态漂移。
- slots=True：字段固定，拼错属性时更容易暴露问题，也节省少量内存。

5.1 ToolCall

```
@dataclass(frozen=True, slots=True)
class ToolCall:
    id: str
    name: str
    arguments: dict[str, JSONValue]
```

它表示模型提出的一次工具调用：

- id 用来把工具结果与本次调用配对；
- name 是工具名，例如 read_file；
- arguments 是已经解析成字典的 JSON 参数。

5.2 ModelReply

ModelReply 把不同模型厂商的响应统一成两个字段：

- content：模型文本；
- tool_calls：零个或多个 ToolCall。

Agent 因此不需要理解 OpenAI SDK 返回对象的内部结构。

5.3 ToolResult

所有工具都返回相同结构：

ok	是否成功
data	成功或失败的结构化数据
error_code	稳定、可供程序判断的错误码
message	给模型和人看的说明
metadata	changed_file、timed_out 等运行时证据

注意 `message` 和 `error_code` 的区别：前者适合阅读，后者适合程序逻辑。比如路径越界时，模型可以收到 `path_outside_workspace`，而不是捕获一个未处理的 Python 异常。

5.4 RunResult

`RunResult` 不是模型自己声称的结果，而是运行时根据事实生成的：

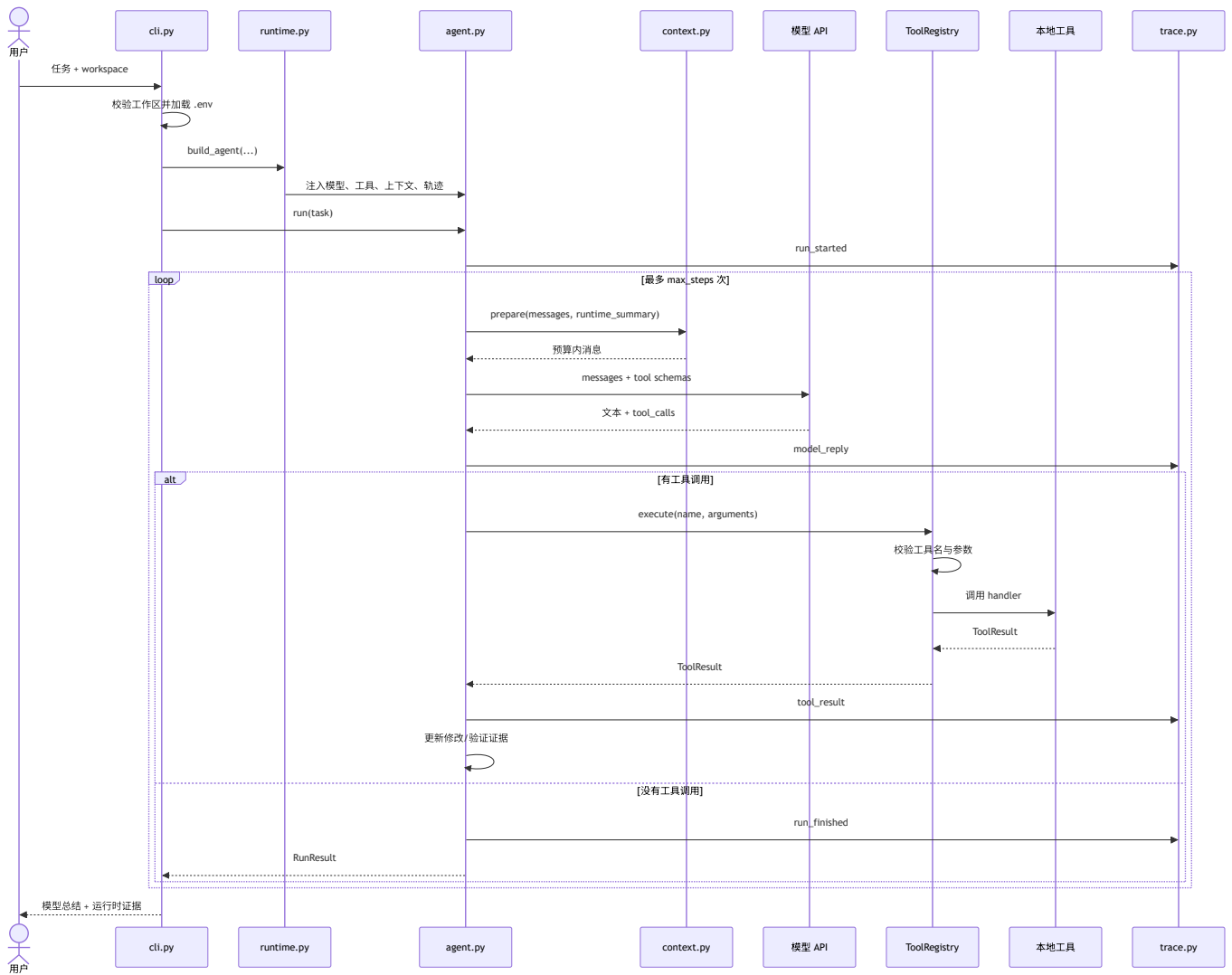
- 最终文本；
- 终止原因；
- 验证状态；
- 已知修改文件；
- JSONL 轨迹路径；
- 使用步数；
- 命令是否可能产生运行时无法完全追踪的副作用。

`successful` 只有在终止原因为 `completed`，且验证状态为 `not_required` 或 `verified` 时才为真。

6. 一次任务的完整生命周期

下面以 CLI 命令为例：

```
tracecoder run "修复失败测试并验证" --workspace .
```



关键点是：一次“模型回复”不等于一次“Agent 运行”。一次 Agent 运行可能包含多次模型回复和多次工具执行。

7. 核心循环： Agent.run()

src/tracecoder/agent.py 是最重要的文件。可以把它简化成下面的伪代码：

```

messages = [system_prompt, *history, current_user_task]

for step in range(max_steps):
    if user_cancelled:
        return interrupted_result

    request_messages = context.prepare(messages, runtime_facts)
    reply = model.complete(request_messages, tool_schemas)
    messages.append(reply_as_assistant_message)

    if reply has no tool calls:
        if files_changed_but_not_verified and reminder_not_sent:
            messages.append(verification_reminder)
            continue
        return completed_result

    for call in reply.tool_calls:
        detect_repeated_call(call)
        result = registry.execute(call.name, call.arguments)
        messages.append(tool_result_message)
        update_runtime_evidence(call, result)

return max_steps_result

```

7.1 构造消息历史

每次运行都从一个固定系统提示词开始，然后加入上一轮可复用消息，最后加入当前用户任务：

```

messages = [
    {"role": "system", "content": SYSTEM_PROMPT},
    *history,
    {"role": "user", "content": task},
]

```

系统提示词要求模型先检查工作区、聚焦修改、处理工具错误，并在修改后使用 `purpose='verify'` 的命令验证。

Web 多轮续聊时，`history` 来自同一会话上一轮保存的 `conversation_messages`；CLI 的单次 `run` 默认没有跨命令持久历史。

7.2 每步都重新计算运行时事实

`_runtime_summary()` 会把下列事实放入上下文：

- 已知修改文件；
- 当前验证状态；
- 命令副作用是否未知；
- 最近五个工具失败。

这些事实来自本地运行时，不依赖模型记忆。即使早期消息被压缩，模型仍然知道当前是否需要验证。

7.3 模型回复与工具调用

`model.complete()` 返回标准化的 `ModelReply`。Agent 先把回复记录到轨迹，再把它转换为 OpenAI-compatible assistant 消息。

如果回复包含工具调用，assistant 消息必须保留：

- 工具调用 ID；
- 工具名称；
- JSON 字符串形式的参数。

随后每个工具结果用同一个 `tool_call_id` 追加为 `role="tool"` 的消息。这个 ID 配对不能丢，否则很多 API 会认为对话历史非法。

7.4 工具按顺序执行

同一模型回复可能请求多个工具。TraceCoder 逐个执行，而不是并发执行。这样更容易：

- 保持结果顺序；
- 在用户点击停止后跳过后续工具；
- 避免两个写工具同时修改同一文件；
- 准确更新验证状态。

7.5 重复调用检测

`_tool_fingerprint()` 使用“工具名 + 排序后的 JSON 参数”生成指纹。如果模型连续多次返回完全相同的调用，`repeat_count` 会增加。达到 `repeat_limit` 后，Agent 以 `repeated_call` 终止。

这里只检测连续相同调用，而不是禁止整个任务中再次读取同一个文件。因为模型修改文件后再次读取它可能是合理行为。

7.6 最大步数

外层循环最多运行 `max_steps` 次，默认值来自配置，默认 20。模型无法通过不断换工具调用来无限运行。达到上限后返回 `max_steps`，并写入结束轨迹。

验证提醒不会额外增加步数预算。只有执行 `continue` 后还剩下一次循环迭代，Agent 才会真的再次请求模型；如果提醒恰好在最后一个允许步骤追加，循环会直接结束并返回 `max_steps`，模型不会再收到一次请求。

7.7 异常不会让进程无说明崩溃

- 到达 `Agent.run()` 的 `KeyboardInterrupt` 会转换为 `interrupted`；
- 模型 API 或协议异常转换为 `provider_error`；
- 工具普通异常由注册表转换为 `execution_error`；
- `ToolRegistry` 会重新抛出 `KeyboardInterrupt` 和 `SystemExit`，不会把它们误包装成 `execution_error`；
- 命令审批是一个有意设计的例外：CLI 的 `make_approval()` 和 `RunCommandTool` 都会捕获审批期间的 `KeyboardInterrupt`，把它当作“拒绝命令”，最终得到 `command_denied`，而不是一定终止整个 Agent 运行。

所以，`Ctrl+C` 的结果取决于中断发生的位置：在命令审批输入期间按下通常只是拒绝该命令；其他位置产生且一路到达 `Agent.run()` 的 `KeyboardInterrupt` 才会使本次运行以 `interrupted` 结束。

最终所有出口都通过 `_finish()` 创建 `RunResult` 并记录 `run_finished`。

8. 修改与验证状态

模型可能说“测试通过了”，但这不等于运行时真的观察到测试。`TraceCoder` 单独维护 `VerificationStatus`：

状态	含义
<code>not_required</code>	当前没有已知文件修改
<code>required</code>	已修改文件，或执行了普通工作命令，需要验证
<code>verified</code>	最新修改之后，成功执行了 <code>purpose="verify"</code> 的命令
<code>failed</code>	验证命令执行失败

文件写入和精确替换成功后，工具结果的 `metadata.changed_file` 会使状态变为 `required`。

命令工具无论成功失败，都标记 `shell_side_effects_unknown=True`，因为任意程序可能修改多个文件，运行时无法仅靠返回值完整推断副作用。

如果模型在 `required` 或 `failed` 状态下直接给最终回答，Agent 会追加一次验证提醒。只要 `max_steps` 还剩迭代，下一步就会再请求模型；提醒只发送一次，防止模型一直拒绝验证而形成无限循环。若下一次最终回答真的发生，它会被接受，但结果仍会明确显示未验证或验证失败；若提醒已用掉最后一步，则按第 7.6 节所述直接以 `max_steps` 结束。

一个重要细节：如果先验证成功，之后又写了文件，状态会重新变回 `required`。旧测试不能证明新修改正确。

9. 工具系统

9.1 为什么需要 ToolRegistry

ToolRegistry 同时保存四样东西：

- 1. 工具名称；
- 2. 给模型看的自然语言描述；
- 3. JSON Schema 参数定义；
- 4. 本地 Python handler。

注册表可以生成 OpenAI-compatible 的工具列表，也能在执行前统一校验参数。这样 Agent 不需要写大量 `if tool_name == ...`。

9.2 六个内置工具

工具	作用	关键限制
<code>list_files</code>	浏览目录，可选递归	结果数量有上限，不跟随目录符号链接
<code>search_text</code>	在 UTF-8 文件中搜索字面文本	单文件读取和结果数量有上限
<code>read_file</code>	读取指定行范围	最多读取受限字节和行数
<code>write_file</code>	创建或覆盖文本文件	父目录必须存在，原子写入
<code>replace_text</code>	精确替换局部文本	实际匹配数必须等于预期值
<code>run_command</code>	执行参数数组形式的命令	用户审批、超时、输出上限、 <code>shell=False</code>

9.3 参数校验

registry.py 实现了一个项目所需的最小 JSON Schema 校验器：

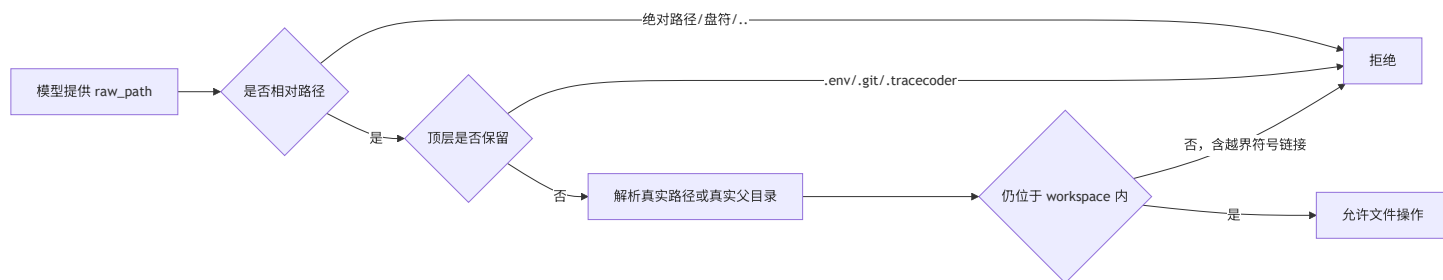
- 检查必填字段；
- 拒绝 additionalProperties=False 下的未知字段；
- 检查 string、integer、number、boolean、array、object；
- 检查枚举、整数上下限、数组最少项数；
- 递归校验数组元素。

校验失败返回 invalid_arguments，handler 不会被调用。未知工具返回 unknown_tool。

这层校验非常重要，因为模型输出只是外部输入，不能因为“通常格式正确”就直接相信。

10. 文件工具的安全边界

WorkspacePolicy 是所有文件工具共享的路径守门员。它首先把用户指定的工作区解析成真实规范路径，然后所有模型路径都必须经过它。



10.1 拒绝的路径

- C:\secret.txt、/etc/passwd 等绝对路径；
- 带 Windows 盘符的路径；
- 包含 .. 的父目录穿越；
- 解析后落在工作区外的符号链接；
- 顶层 .env、.git、.tracecoder。

.env.example 不是保留名，因此可以作为配置模板读取。

10.2 为什么写文件也要解析父目录

新文件还不存在，不能对它执行 resolve(strict=True)。所以 resolve_for_write() 会解析它已经存在的父目录，确认父目录在工作区内，再拼接文件名。父目录不存在时直接拒绝，避免隐式创建一长串未

知目录。

10.3 原子写入

`_atomic_write()` 的步骤是：

1. 在目标文件同目录创建临时文件；
2. 写入 UTF-8 字节；
3. `flush` 并 `fsync`；
4. 使用 `os.replace` 一次性替换目标；
5. 异常时清理临时文件。

这比直接打开目标文件覆盖更可靠。若写到一半进程崩溃，旧文件通常仍完整，不容易留下半截内容。

10.4 精确替换

`replace_text` 要求模型提供 `expected_replacements`。例如模型认为某段文本只出现一次，但实际出现了三次，工具会拒绝修改并返回实际匹配数。它避免一个模糊替换误改多个位置。

11. 命令执行与审批

`RunCommandTool` 接收的是参数数组：

```
{
  "argv": ["python", "-m", "pytest", "-q"],
  "purpose": "verify",
  "timeout_sec": 120
}
```

它不是一段默认交给 Shell 解析的字符串。核心调用是：

```
subprocess.Popen(
    argv,
    cwd=workspace,
    env=minimal_environment,
    shell=False,
    ...
)
```

这意味着 `|`、`&&`、重定向符号等不会被隐式解释。确实需要 Shell 时，模型必须明确请求 `cmd /c ...` 或 `sh -lc ...`，用户也能在审批时看到真实参数。

11.1 审批发生在启动进程之前

CLI 显示精确 `argv` 和工作目录；Web 后端创建唯一 `approval_id`，页面显示中文用途说明，并把完整参数折叠在“查看完整命令”中。

只有当前仍待处理且 ID 完全匹配的审批才能被接受。过期 ID、错误 ID 和重复提交都会返回冲突，避免用户批准了旧命令却错误放行新命令。

CLI 等待输入时，如果用户按 `Ctrl+C`（或输入流结束），审批回调返回拒绝；`RunCommandTool` 自己也有同样的兜底。因此进程尚未启动，工具结果是 `command_denied`，Agent 可以把这条失败结果交回模型继续处理。

11.2 子进程环境最小化

命令只继承允许列表中的常见系统变量，例如 `PATH`、`TEMP`、`SYSTEMROOT`、`VIRTUAL_ENV`。API key 不在允许列表中，所以子进程默认无法从环境变量读取模型凭据。

11.3 输出上限与超时

`stdout` 和 `stderr` 由两个后台线程同时读取，避免某一管道填满导致子进程阻塞。每个流只保留配置允许的字节数，但线程仍继续排空剩余输出，因此“大量输出”不会轻易造成死锁。

超时后调用 `process.kill()` 结束直接进程，并返回 `command_timeout`。这是工程上的边界而不是完整沙箱：被批准的命令可能访问工作区外资源，也可能产生无法全部回收的后代进程。

12. 模型配置与适配器

12.1 .env 加载

`Settings.from_env()` 先读取工作区 `.env`，再合并进程环境变量。这里不能简单理解成一条覆盖所有别名的优先级链，而要分两步看：

同名键合并：进程环境变量 > 工作区 `.env`

三组兼容别名的选择（合并后）：`TRACECODER_*` > `OPENAI_*`

也就是说，进程中的 `TRACECODER_MODEL` 会覆盖 `.env` 中同名的 `TRACECODER_MODEL`；但如果 `.env` 定义了 `TRACECODER_MODEL`，进程只定义 `OPENAI_MODEL`，合并后代码仍会选择前者。为避免这种跨来源、跨

别名的意外组合，建议整个项目统一使用 `TRACECODER_*` 这一组名称，不要与 `OPENAI_*` 混用。

必需配置：

- `TRACECODER_API_KEY`，也兼容 `OPENAI_API_KEY`；
- `TRACECODER_MODEL`，也兼容 `OPENAI_MODEL`。

API 根地址也兼容 `OPENAI_BASE_URL`：代码依次选择 `TRACECODER_BASE_URL`、`OPENAI_BASE_URL`，两者都没有时才使用默认 OpenAI 地址。最大步数、重复阈值、上下文字符预算、命令超时和输出上限只使用各自的 `TRACECODER_*` 环境变量配置。

`api_key` 和 `redaction_secrets` 使用 `repr=False`，避免打印 `Settings` 时意外显示凭据。

12.2 Provider-neutral 协议

`llm/base.py` 只定义一个 `ModelClient Protocol`：

```
def complete(messages, tools) -> ModelReply:
    ...
```

Agent 只依赖这个协议。真实实现 `OpenAIBCompatibleClient` 调用 Chat Completions；测试实现 `FakeModelClient` 返回预设回复。

12.3 OpenAI-compatible 适配器

适配器负责：

1. 把模型名、消息和工具定义交给 SDK；
2. 取出第一条 assistant 消息；
3. 解析每个工具调用的 JSON 参数；
4. 确保参数是对象而不是数组或字符串；
5. 检查工具调用 ID 和名称；
6. 转换成项目自己的 `ModelReply`。

提供方返回畸形结构时抛出 `ModelProtocolError`，随后 Agent 把它转换成可见的 `provider_error` 结果。

这里还有一个重要的提供方兼容边界：当前 `ModelReply` 以及适配器只保留 `content` 和 `tool_calls`，Agent 重建 assistant 历史消息时也只有这两个字段，不会保存提供方额外返回的 `reasoning_content`。

DeepSeek V4 thinking mode 官方说明要求：请求包含 `tools` 时，后续请求必须完整回传以前各轮的 `reasoning_content`。因此，在适配器支持该字段，或请求能够明确关闭 thinking mode 之前，不应把当前实现视为完整兼容 DeepSeek V4 thinking-mode 工具调用；这类多步骤任务可能在第一次工具结果之后的下一次模型请求失败。这个限制针对 thinking mode，不代表所有 OpenAI-compatible 提供方或非 thinking 请求都会失败。

13. 上下文管理： ContextManager

多轮会话和长工具输出会让消息越来越大。TraceCoder 用字符预算做确定性压缩，不再调用另一个模型总结，因此结果可测试、没有额外费用。

13.1 未超预算

消息序列化后的字符数不超过 `max_chars` 时，返回一个浅拷贝，内容和顺序不变。

13.2 超预算时必须保留什么

- 系统提示词；
- 当前用户问题；
- 本地生成的运行时事实；
- 尽可能多的最近完整对话轮次；
- 当前轮尽可能多的最近工具调用包。

“工具调用包”指 assistant 的 tool call 与紧随其后的全部 tool result。不能只保留其中一半。

13.3 多轮压缩

项目先找到最新的 `role="user"` 消息，把更早内容视为历史轮次，把后面内容视为当前轮执行过程。它从最近历史开始向前选择完整轮次，候选消息一旦超预算就停止。

如果当前轮最新工具结果本身很大，`_compact_bundle()` 会保留角色、工具 ID、函数名和结构，但把大段内容替换为明确的截断标记。若连“系统提示 + 运行事实 + 当前问题”都超过预算，则抛出错误，而不是悄悄返回一个超预算请求。

13.4 会话记忆不等于永久存储

Web 后端把每个会话的可复用消息保存在当前服务进程内存中。用户继续在同一对话输入时，这些消息会传给新 Agent。点击“新对话”会提交空 `conversation_id`，后端创建独立会话。

当前实现保留最近 50 个会话，但服务重启后历史消失。上下文压缩解决“单次模型请求不能无限大”，并不等于数据库持久化。

14. 可追踪执行： TraceRecorder

每次 Agent 运行都会在工作区 `.tracecoder/traces/<session-id>.jsonl` 生成追加式轨迹。

JSONL 表示“一行一个 JSON 对象”。每条事件包含：

- `session_id` ；
- 单调递增的 `seq` ；
- UTC 时间；
- `event_type` ；
- 已脱敏的 `payload` 。

常见事件有：

- `run_started` ；
- `model_reply` ；
- `tool_requested` ；
- `tool_result` ；
- `verification_reminder` ；
- `approval_required` / `approval_resolved` ；
- `cancel_requested` ；
- `run_finished` 。

`redact()` 会递归处理字符串、列表、元组和字典，把已配置 API key 子串替换成 `[REDACTED]`。工具结果在写入轨迹前脱敏，发送回模型的结果也使用脱敏版本。

`TraceRecorder` 使用可重入锁保护序号和文件写入，因此 Web 控制事件与 Agent 事件并发记录时仍能保持合法顺序。

CLI 的 `trace` 子命令使用 `read_trace()` 逐行解析。缺失文件、损坏 JSON 或缺少事件类型都会给出明确错误。

15. 运行时装配与 CLI

15.1 runtime.py

`build_agent()` 是 CLI 和 Web 的共同“装配工厂”。它创建：

- 1. TraceRecorder ；
- 2. 六工具 ToolRegistry ；
- 3. OpenAICompatibleClient ；
- 4. ContextManager ；
- 5. 注入以上依赖的 Agent 。

审批回调、事件观察回调、取消回调和会话 ID 都从入口层注入。Agent 因此不知道自己运行在终端还是浏览器中。

15.2 cli.py

CLI 有三个子命令：

命令	用途
tracecoder run	运行一次自然语言编程任务
tracecoder trace	格式化查看 JSONL 轨迹
tracecoder web	启动本地 Web GUI

run 的启动顺序是先验证任务和工作区，再加载配置，最后创建 Agent。错误配置不会在已经执行副作用后才被发现。

CLI 输出分成两部分：

- “Model summary”：模型最终文本；
- “Runtime evidence”：终止原因、验证状态、修改文件、未知命令副作用和轨迹路径。

这避免把“模型说完成了”和“运行时证据表明已验证”混成一件事。

web 默认只允许回环地址。绑定 0.0.0.0 等非回环地址时必须显式提供 --trust-proxy-auth ，表示调用者确认前方存在已经认证的代理，例如私有 Codespaces 端口。这个参数本身不实现登录。

16. Web 后端： web.py

Web 层使用 FastAPI，但 FastAPI 只负责 HTTP 和数据校验。真正的 Agent 决策仍在 agent.py 。

16.1 三个核心状态对象

- _ManagedConversation ：会话 ID、标题、该会话的 run ID 列表、可复用消息历史；
- _ManagedRun ：某一轮任务、附件、上一轮消息、当前状态、事件、结果、审批和取消信号；

- `_PendingApproval`：审批 ID、精确 `argv`、`cwd`、等待事件和最终决定。

一个 `conversation` 可以包含很多 `run`。Web 页面中的“一轮”对应一个 `_ManagedRun`。

16.2 `RunManager.start()`

启动任务时：

1. 拒绝空任务；
2. 检查是否已有活动任务；
3. 没有 `conversation_id` 就创建新会话；
4. 有 ID 就读取原会话，否则返回 404；
5. 把该会话移动到最近使用队列末尾；
6. 超过 50 个会话时淘汰最旧的完整会话；
7. 创建 `run`，复制会话消息为 `prior_messages`；
8. 在后台线程执行 Agent。

同一工作区只允许一个活动任务，这是因为文件和命令都作用于同一个共享目录。两个 Agent 同时修改文件会让审批、验证和最终证据变得难以解释。

16.3 多轮上下文如何落地

后台线程发现 Agent 实现了 `ConversationalRunnableAgent` 时，会执行：

```
result = agent.run(current_task, history=managed.prior_messages)
conversation.messages = agent.conversation_messages
```

所以第二轮模型请求实际包含第一轮的用户、助手，以及完整工具调用包。运行结束时，系统提醒、孤立工具结果和被取消后未完成的工具包会被 `_reusable_messages()` 清理，避免把非法消息历史带到下一轮。

16.4 状态与锁

`RunManager` 有一个全局锁，保护会话字典、运行字典和唯一活动 ID；每个 `_ManagedRun` 还有自己的锁，保护状态、事件、审批和结果。

运行结束时，结果写入、会话消息更新和活动 ID 释放在同一个协调区间完成。否则浏览器可能在第一轮历史尚未保存时立即启动第二轮，导致第二轮丢失上下文。

16.5 命令审批

当命令工具调用审批回调时，后台线程：

1. 创建新的 `approval_id`；
2. 把 `run` 状态改为 `waiting_approval`；
3. 记录审批事件；
4. 每 0.1 秒等待页面决定，同时检查取消信号；
5. 决定后恢复 `running` 或拒绝命令。

页面只提交审批 ID 和布尔决定。后端再次核对它是否仍是当前审批。

中文命令说明由 `_describe_command()` 本地确定性生成。它识别编译器、Python、pytest、Git、前端包管理器和新建目录等常见命令；完整 `argv` 始终单独保留，因此易读说明不会替代真实审批对象，也不会额外调用模型。

16.6 协作式取消

点击停止不会粗暴杀死正在执行的模型 HTTP 请求。它会设置 `cancel_event`：

- 如果正在等待审批，立即把审批决定设为拒绝并解除等待；
- Agent 在调用模型前、模型返回后、每个工具前都会检查取消状态；
- 当前无法中断的外部调用返回后，循环以 `interrupted` 结束。

这是“协作式取消”：边界清楚、状态可追踪，但不承诺瞬间打断任意第三方网络调用或已批准子进程。

16.7 文件上传

`POST /api/uploads` 只接受 `application/octet-stream`，边流式读取边检查 10 MiB 上限。文件名必须是安全 `basename`，拒绝路径分隔符、控制字符、Windows 保留设备名和末尾空格/点。

文件保存到工作区 `uploads/`：

- 目录不能是符号链接；
- 同名文件不覆盖，而是生成 `name-2.ext`；
- 使用排他创建 `xb` 避免竞态覆盖；
- 提交任务时再次确认附件仍是 `uploads/` 内普通文件。

Agent 收到的是普通工作区相对路径，所以后续仍通过同一套文件工具读写。

16.8 HTTP API

方法与路径	作用
GET /	返回 Web 页面
GET /api/config	返回不含 key 的工作区、提供方和模型
POST /api/runs	启动新一轮或续接会话
GET /api/runs/active	页面刷新后找回活动任务
GET /api/runs/{id}?after=N	增量轮询状态与新事件
POST /api/runs/{id}/approval	处理当前命令审批
POST /api/runs/{id}/cancel	请求停止
GET /api/conversations	最近会话摘要
GET /api/conversations/{id}	某会话全部可见轮次
POST /api/uploads?filename=...	上传附件

17. Web 前端

前端没有 React、Vue 或构建工具，只有原生 HTML、CSS 和 JavaScript。这样依赖少、打包简单，也更容易向评委展示状态管理逻辑确实由项目自行实现。

17.1 index.html

HTML 定义：

- 顶部运行状态；
- 左侧模型、新对话按钮和历史列表；
- 会话标题与停止按钮；
- 首屏问候；
- 命令审批卡；
- 固定在底部的输入框和附件按钮；
- 用户消息、模型消息、折叠过程事件三个 template。

JavaScript 通过 clone template 创建消息，避免拼接大段 HTML 字符串。

17.2 styles.css

页面使用两列网格：固定宽度侧栏和自适应聊天区。聊天区内部再用“头部、可滚动时间线、审批卡、输入区”网格布局，因此输入框保持在底部，只有消息时间线滚动。

媒体查询在窄屏下把侧栏和聊天区改成上下布局。Markdown、代码块、列表、引用、审批和过程卡都有独立样式。

17.3 浏览器状态

app.js 的 state 是前端单一状态源，重要字段包括：

- conversationId：当前显示会话；
- runId：当前显示轮次；
- activeRunId：服务器仍在执行的轮次；
- lastEventId：已渲染到的事件游标；
- pollingGeneration：废弃旧轮询；
- navigationGeneration：废弃旧页面请求；
- approvalId：当前审批；
- attachments：尚未提交的附件；
- conversations：左侧历史摘要。

17.4 提交与轮询

提交任务时，前端把当前 conversation_id 一并发送：

- 值为现有 ID：后端续接同一会话；
- 值为 null：后端创建新会话。

后端任务在后台线程运行，前端每 650 ms 请求一次增量状态。请求携带 after=lastEventId，服务器只返回新事件，避免每次重复传输全部轨迹。

终态后，前端停止轮询、清除审批，并单独渲染最终回答。

17.5 为什么最终回答不属于“工具过程”

模型的最后一次无工具调用回复先以 model_reply 事件出现，RunResult 可能在下一次轮询才可见。前端暂存“可能是最终回复”的事件：

- 若后面还有其他事件，说明它只是中间输出，加入折叠过程；
- 若运行结束且内容等于 result.final_text，它作为最终回答展示，不重复放进过程。

因此用户默认看到清晰最终答案，模型中间输出、工具请求和结果折叠在“思考与工具过程”中。这里展示的是可观察的模型文本和工具事件，不等同于模型内部不可见的私有推理过程。

17.6 安全 Markdown

`markdown.js` 是一个小型、无依赖渲染器，支持标题、段落、粗体、斜体、删除线、行内代码、代码块、链接、列表、引用和分隔线。

它先转义 `& < > " ' ``，再添加允许的 Markdown 标签。链接只接受 `http`、`https`、`mailto`、相对路径、绝对站内路径和锚点，拒绝 `javascript:`、`data:`、`vbscript:`。

用户消息和工具过程使用 `textContent`；只有最终 `assistant` 文本经过安全渲染后写入 `innerHTML`。

17.7 首屏问候与新对话

`resetTimeline()` 恢复首屏问候并把标题改为“新对话”。一旦追加任意消息或过程卡，问候就隐藏。

点击“新对话”时：

1. 活动任务存在则拒绝切换；
2. 增加导航代数，令旧异步响应失效；
3. 停止旧轮询；
4. 清空 `conversationId`、`runId` 和附件；
5. 恢复空时间线；
6. 不删除左侧历史。

所以新对话是用户显式动作，而不是每次回答结束后自动发生。

17.8 异步竞态处理

浏览器中多个请求可能乱序返回。例如用户点击历史 A，随后立刻点“新对话”，历史 A 的响应可能最后才到。如果直接渲染，它会覆盖新对话。

`navigationGeneration` 类似一个版本号：每次导航或提交先递增，请求完成后比较旧版本与当前版本，不一致就丢弃响应。同样，上传开始后若用户切换页面，文件仍安全保存，但不会错误附加到新会话。

如果 `POST /api/runs` 已被服务器接受，但网络在返回响应前断开，前端会请求 `/api/runs/active` 尝试找回已启动任务，避免同一任务重复提交。

18. 测试如何证明 Agent 正确

测试不依赖真实 API key，也不发送网络请求。

18.1 FakeModelClient

假模型接收一组预设 `ModelReply`，每次 `complete()` 返回下一条，并把收到的 `messages` 保存到 `requests`。

例如测试可以安排：

- 1. 第一次回复请求 `read_file`；
- 2. 第二次回复请求 `write_file`；
- 3. 第三次回复请求 `run_command purpose=verify`；
- 4. 第四次回复给最终文本。

然后断言文件真的改变、模型真的收到每个工具结果、验证状态为 `verified`、轨迹以 `run_finished` 结束。

这比只测试几个辅助函数更有价值，因为它覆盖了完整跨层链路。

18.2 测试分层

测试文件	主要证明
test_config.py	配置优先级、必填项、正整数、脱敏候选
test_registry.py	未知工具和错误参数不会执行 handler
test_filesystem_tools.py	路径越界、保留目录、符号链接、原子写入、精确替换
test_shell_tool.py	拒绝不启动、无隐式 Shell、超时、输出限制、key 不进入子进程
test_context.py	小历史不变、超预算压缩、完整工具包、多轮当前问题
test_trace.py	JSONL、脱敏、损坏输入、并发顺序
test_openai_adapter.py	多工具调用解析和畸形参数拒绝
test_agent_loop.py	读写验证闭环、重复/步数、错误、取消、多轮历史
test_cli.py	参数、工作区、dotenv、Web 非回环保护
test_web.py	会话、上传、事件、审批、取消、淘汰与真实 Agent 集成

测试文件	主要证明
markdown_smoke.cjs	Markdown 功能与 XSS 转义
web_ui_smoke.cjs	执行实际前端脚本，验证问候、多轮、新对话和异步恢复

18.3 GitHub Actions

`.github/workflows/ci.yml` 在四种组合运行：

- Ubuntu + Python 3.11；
- Ubuntu + Python 3.13；
- Windows + Python 3.11；
- Windows + Python 3.13。

每个任务会：

1. 安装项目与开发依赖；
2. 构建 wheel；
3. 检查 Web 静态文件确实进入 wheel；
4. 检查并执行 JavaScript 冒烟测试；
5. 运行 pytest；
6. 运行 Ruff；
7. 运行严格 mypy。

这不仅证明“本机能跑”，还证明路径、进程和编码相关逻辑能跨 Windows/Linux 工作。

19. 用一个具体例子串起全部代码

假设用户发送：“读取 `calculator.py` ，修复加法错误并运行测试。”

1. CLI/Web 把任务交给 `Agent.run()` 。
2. Agent 发送系统提示、任务和六个工具 Schema。
3. 模型请求 `read_file({"path": "calculator.py"})` 。
4. `ToolRegistry` 校验参数；`WorkspacePolicy` 确认路径在工作区；工具读取文件。
5. 结果作为 `role="tool"` 消息返回模型。
6. 模型请求 `replace_text` ，并声明预期只匹配一次。
7. 替换成功，`metadata` 记录 `changed_file="calculator.py"` ，验证状态变为 `required` 。
8. 模型请求 `run_command(["python", "-m", "pytest", "-q"], purpose="verify")` 。
9. 用户看到命令和 `cwd`，点击批准。

10. 子进程在工作区运行，不继承 API key；stdout/stderr 被限长捕获。
11. 命令退出码为 0，验证状态变为 `verified`。
12. 模型返回无工具调用的最终说明。
13. Agent 返回 `RunResult`，其中含修改文件、验证状态、终止原因和 `trace` 路径。
14. Web 把工具过程折叠，最终说明按安全 Markdown 展示。

其中每一步都能在 JSONL 轨迹或测试中找到证据。

20. 如何运行和调试

20.1 安装

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -e ".[dev]"
```

根目录创建 `.env`：

```
TRACECODER_API_KEY=你的密钥
TRACECODER_BASE_URL=https://api.provider.example/v1
TRACECODER_MODEL=支持原生工具调用的模型名
```

上面的域名是提供方无关的占位示例，需要替换为你实际使用的 OpenAI-compatible API 根地址；使用默认 OpenAI 地址时可以省略 `TRACECODER_BASE_URL`。若使用 DeepSeek V4，还要先阅读第 12.3 节的 `thinking-mode` 工具调用限制。

20.2 启动 CLI

```
.\.venv\Scripts\tracecoder.exe run "阅读项目并介绍代码结构" --workspace .
```

20.3 启动 Web

```
.\.venv\Scripts\tracecoder.exe web --workspace .
```

浏览器访问 `http://127.0.0.1:8765`。按 `Ctrl+C` 停止服务。

20.4 查看轨迹

```
.\.venv\Scripts\tracecoder.exe trace .\tracecoder\traces\<session-id>.jsonl
```

20.5 运行开发检查

```
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe -m ruff check .
.\.venv\Scripts\python.exe -m mypy
node tests\markdown_smoke.cjs
node tests\web_ui_smoke.cjs
```

调试 Agent 时，优先从 trace 中按 seq 查看“模型回复 -> 工具请求 -> 工具结果 -> 状态变化”，不要只看最后一段回答。

21. 如何扩展项目

21.1 新增一个工具

1. 在 tools/ 中实现返回 ToolResult 的 handler；
2. 在 build_tool_registry() 中注册名称、描述和 JSON Schema；
3. 确定是否产生文件修改，并在 metadata 中提供 changed_file；
4. 为正常、错误参数和边界场景补测试；
5. 若工具有高风险副作用，设计独立审批，而不是默认复用文件工具权限。

Agent 循环通常不需要修改，因为它只依赖注册表统一合同。

21.2 支持另一种模型 API

实现 ModelClient.complete()，把厂商响应转换为 ModelReply。不要把新 SDK 对象泄漏到 domain.py 或 agent.py。

21.3 实现历史持久化

当前 _ManagedConversation 在内存中。可以增加一个 ConversationStore 接口，把 conversation 元数据和可复用 messages 保存到 SQLite。需要特别处理：

- 工具调用 ID 和 tool result 的完整配对；
- 服务重启时处于 running/waiting_approval 的旧任务应恢复为什么状态；

- trace 与 conversation 的关联；
- 数据库大小和历史淘汰；
- API key 与敏感工具输出是否允许长期保存。

21.4 实现流式模型输出

当前模型调用同步返回完整消息。流式输出需要在适配器、Agent 事件、Web API 和前端渲染四层共同设计，不能只把 SDK 改成 stream。尤其要区分“尚未完整的文本片段”和“已经完整、可以执行的工具参数”。

22. 当前限制

- Web 会话仅保存在当前进程内存中，重启后消失。
- 同一工作区只允许一个活动任务，不支持多 Agent 并发。
- 模型请求与工具执行是同步的；停止采用协作式取消。
- 命令审批不是沙箱，批准命令仍具有当前用户权限。
- 超时保证终止直接进程，不保证所有后代进程都消失。
- 文件工具面向最大 1 MiB 的文本读取，不适合大型二进制文件。
- OpenAI-compatible 适配器尚未保留 reasoning_content ，因此尚不能可靠完成 DeepSeek V4 thinking-mode 的多步骤工具调用。
- Markdown 渲染器只实现常用子集，不是完整 CommonMark。
- 没有账号系统；Web 默认只应在本机或受认证代理保护的私有端口使用。
- 没有完整网页 IDE，文件编辑依靠 Agent 工具或 Codespaces/本地编辑器。

这些限制并不意味着设计失败。一个好的个人项目应该明确告诉读者“它保证什么、没有保证什么”。

23. 初学者词汇表

词语	在本项目中的含义
Agent / 智能体	能循环调用模型和工具、根据结果继续行动的程序
Harness / 运行时	包围模型的本地控制代码，包括历史、工具、循环、终止和错误处理
Tool calling	模型以结构化数据请求调用函数的协议
JSON Schema	描述工具参数名称、类型和限制的结构
Adapter / 适配器	把外部 SDK 响应转换成项目内部统一类型

词语	在本项目中的含义
Workspace / 工作区	文件工具允许读写的根目录，也是命令默认 cwd
cwd	子进程的当前工作目录
argv	传给进程的参数数组，第一项通常是可执行程序
shell=False	参数直接交给程序，不自动解释管道、重定向等 Shell 语法
Context / 上下文	一次模型请求携带的系统、用户、assistant 和 tool 消息
Compaction / 压缩	超预算时保留关键结构并删除较早或过大的内容
JSONL	一行一个 JSON 对象的日志格式
Trace / 轨迹	一次运行的模型、工具、审批、耗时和结束事件
Polling / 轮询	浏览器定期请求后端新状态
Race condition / 竞态	多个异步操作完成顺序不同导致状态被旧结果覆盖
Protocol	只规定对象需要有哪些方法，不绑定具体实现
Fake / Test double	测试中替代真实模型的可控对象

24. 推荐阅读代码顺序

如果你准备答辩或自己讲解项目，建议按下面顺序打开文件：

- 1. src/tracecoder/domain.py：先认识数据。
- 2. src/tracecoder/llm/base.py 和 tests/fakes.py：理解“可替换模型”。
- 3. src/tracecoder/tools/registry.py：理解工具如何定义和校验。
- 4. src/tracecoder/tools/filesystem.py：理解工作区安全。
- 5. src/tracecoder/tools/shell.py：理解命令审批和执行。
- 6. src/tracecoder/agent.py：带着工具知识阅读核心循环。
- 7. src/tracecoder/context.py 与 trace.py：理解长历史与证据。
- 8. src/tracecoder/runtime.py 与 cli.py：理解如何组装并启动。
- 9. src/tracecoder/web.py：理解会话、线程、审批和 API。
- 10. web_static/app.js：理解浏览器如何显示后端状态。
- 11. tests/test_agent_loop.py 与 tests/test_web.py：用测试反向确认设计。

最终可以用一句话概括 TraceCoder：

它把大语言模型提出的结构化工具请求，转换为经过校验、审批、限制、追踪和验证的本地编程操作，并在有界循环中把结果继续交给模型，直到得到最终答案或确定性停止。